(Sisqual) Greedy Heuristic for Multi-Competence / Multi-Level Scheduling

October 13, 2026

1 Introduction

This document describes a greedy constructive heuristic for the hour-based, multi-competence, multi-level employee scheduling problem defined by the Sisqual specification (*MathematicalDefinition7*). Unlike a pure ILP formulation, the program does not search for a single provably optimal schedule; instead it builds a daily block schedule for every employee through a sequence of local, greedy decisions, assigning a block whenever this is possible, i.e. whenever it does not violate any hard constraint.

Each employee already carries, for every day of the target month, a *marker* inherited from input data (an exact number of contracted hours, a fixed `EQUALS:` time range, a swappable day off, or an unavailability marker such as vacation, sick leave, or a forced day off). The heuristic therefore does not decide *whether* an employee works on a given day, but it decides, for every employee/day pair that is still open and whenever an assignment is possible:

- **when** inside the day the contracted block of hours starts and ends, unless the marker already fixes the time range (`EQUALS:`), and
- **which competence** the employee performs in each half-hour slot of that block, among the competences they are qualified for.

The two objectives that drive every local decision are strictly hierarchical: **Objective 1** (minimise total coverage shortage against the minimum staffing requirements θ_{dtc}) always dominates; **Objective 2** (minimise the sum of competence/level priority weights $p_{c\ell}$ of the choices actually made) is only used to break ties between solutions that reach the same shortage.

Scope : The heuristic described in this document targets exclusively the **minimum** staffing requirements θ_{dtc} referenced in Objective 1. The Sisqual demand specification additionally defines, for every (dtc), an *ideal* staffing level and an *estimated* staffing level, both greater than or equal to the corresponding minimum. Neither of these two additional thresholds is read, tracked, or optimised by any part of the construction process described in this document.

The program executes, per run, 2 conceptual stages:

1. **Setup (`__init__`)**: parses the input files; builds the half-hour slot grid, the per-employee/day candidate block lists, the day-mode classification, and the mutable shortage table `need`;
2. **Greedy construction (`Minimuns`)**: iterates over days and, within each day, over employees in a rarity-driven order, fixing the best feasible block for each employee and updating the shared shortage state;

2 Indexes and Sets

Index / Set	Description
E	Set of employees
D	Set of all days (e.g. `2025-10-01` .. `2025-10-31`)
C	Set of skills/competencies, (`Storage, Checkout, ...`)
L	Set of competence levels present in the problem, (`1,2,3, ...`)
T	Ordered set of half-hour slots covering the full operating window, (`570,600,630 ...`)
$Open(D) \subseteq D$	Days with demand data
$Closed(D) \subseteq D$	Days with no demand data ($D \setminus Open(D)$),
OD	Ordered list of all days in D, according to the `day_order_mode` parameter (closed days are traversed but skipped, since no employee can work on them)
OE_d	Ordered list of employees for day d: employees are grouped by rarity and shuffled within each group
B_{ed}	Set of candidate half-hour blocks for employee e on day d
S_{ed}	Set of scored candidate half-hour blocks for employee e on day d

Table 1: Indexes and sets used throughout this document.

3 Parameters

Parameter	Description
θ_{dtc}	Minimum staffing requirement for day d, slot t, competence c
$contractHours_e$	Contracted daily hours of employee e
$competenceLevels_{ec}$	Competence level $\ell \in L$ of employee e in competence c
$assignableCompetence_e$	Competences employee e can be assigned to
R_{min}	Minimum inter-day rest, in hours, (default 11h)
$p_{c\ell}$	Priority weight of competence c / level ℓ: $p_{c\ell} = 1+(priority-1)\cdot 10$
$marker_{ed}$	Raw schedule marker for (ed): a number of hours, `A`, `DO`, `FDO`, `VAC`, `NOT`, `MED`, `EQUALS:hh:mm-hh:mm`, or empty
$mode_{ed}$	Day mode derived from $marker_{ed}$: `closed` (day without demand), `unavailable` (`FDO`, `VAC`, `NOT`, `MED` or empty marker), `preferred_day_off` (`DO`), `fixed_time_work` (`EQUALS`), `work_template` (number of hours or `A`)
$hours_{ed}$	Hours of the block of employee e on day d: the number written in $marker_{ed}$, or $contractHours_e$ if the marker is `A`; undefined in the other modes

Table 2: Parameters used throughout this document.

4 State Variables

Variable	Description
$need_{dtc}$	Remaining uncovered minimum for (dtc); initialised as a copy of θ and decremented as blocks are fixed
$block_{ed}$	Block chosen for employee e on day d (or `None`)
$competence_{edt}$	Competence actually worked by employee e on day d, slot t

Table 3: State variables used throughout this document.

5 Program Flow

At the highest level the heuristic is a nested loop: an **outer loop over days**, and, for each day, an **inner loop over employees**. Every iteration of the inner loop ends with a single decision, the **schedule selection** step, that fixes a working block for that employee on that day. Algorithm 1 (`Minimuns`, Section 7) is the formal statement of this loop; the rest of this section explains, in prose, how each of its three parts works.

5.1 Day Iteration Order

Three interchangeable day-order strategies are available (Randomized, Demand-Driven, Bottleneck-Driven) and are selected via a parameter of `Minimuns`; they are described individually in Section 6.1.

5.2 Employee Iteration Order

Within a fixed day d, employees are also not visited in an arbitrary order. Two ordering mechanisms combine:

1. a **base ordering** that groups employees by how many competences they can perform and by how rare those competences are across the workforce. Employees with fewer, scarcer competences are placed first, since they have the fewest days on which they can ever be useful for a given competence;
2. a **random shuffle applied within each rarity group**, redone for every day, so that employees who are equally constrained do not always compete for slots in the same fixed order, neither from one day to the next nor across different runs.

Example :

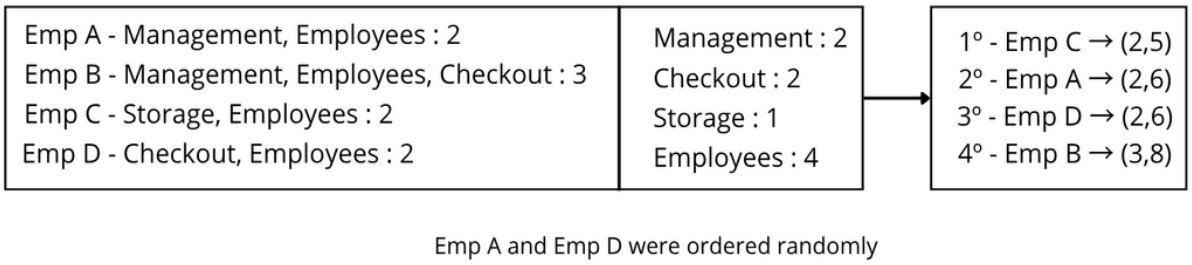


Figure 1: Ordered employees

5.3 Schedule Selection

Once it is an employee's turn on a given day, the heuristic must decide *if*, *when*, and *doing what* that employee works. This happens in four steps:

1. Eligibility check. The employee's day mode $mode_{ed}$ (Table 2) is inspected first. If it is `closed`, `unavailable`, or `preferred_day_off`, the employee is simply skipped for that day, this means there is nothing to select.

2. Candidate block generation. What happens next depends on the employee's marker $marker_{ed}$ (Table 2):

- **Fixed time (`EQUALS:hh:mm-hh:mm`).** The marker already states the working hours of the employee on that day, so the time range is given and nothing is decided about *where* the block sits. B_{ed} contains that single block.
- **Otherwise** (a number of hours). The marker only fixes *how many* hours the employee works, $hours_{ed}$: the number written in the marker. Where that block sits in the day is what the heuristic decides. B_{ed} enumerates every half-hour-aligned starting position for a block of that length, e.g. `08:30-17:30`, `09:00-18:00`, ...

In both cases, the competence performed in each slot of the block is still chosen by the heuristic (steps 3 and 4).

3. Block scoring. Every candidate block in B_{ed} is scored by summing, over each half-hour slot it covers, the shortage contribution of the best-fitting competence for that slot, chosen by the procedure described in Section 6.3, together with that competence's priority-weight cost (Table 2). Formally, for a block $b \in B_{ed}$ covering slots $t \in b$:

$$\text{coverage}(b) = \sum_{t \in b} need_{dt, c^*(t)}, \qquad \text{cost}(b) = \sum_{t \in b} p_{\,c^*(t),\, competenceLevels_{e,c^*(t)}} \tag{1}$$

where $c^*(t)$ is the competence selected for slot t. The result is one $(\text{coverage}(b), \text{cost}(b))$ pair per candidate block, used to rank B_{ed} in the next step.

4. Ordering and application. Candidates are ranked by their coverage first, then lowest priority cost, then random, and the heuristic walks down that ranked list, applying the first block that satisfies the minimum-rest constraint R_{min}, i.e. the **11-hour transition rule** between this block and whatever is already fixed on the neighbouring days for that employee. The first rest-feasible candidate is applied. If every candidate violates the rest rule (or none exist), the employee is left unassigned for that day.

This four-step cycle repeats for every employee on every day until D is exhausted.

6 Function Modules

This section documents, module by module, the functions referenced in Section 5. Each subsection corresponds to one decision point of the schedule-selection cycle.

6.1 Day-Order Modes

Three modes decide the sequence in which D is traversed in Section 5.1. The mode is a parameter of `Minimuns`, so the best-performing mode for a given instance can be found empirically.

6.1.1 Day mode 1 (Randomized)

The simplest strategy: D is shuffled uniformly at random and processed in that order. It carries no information about demand or staffing pressure, but, being random, it provides diversity against which the more informed modes below can be compared.

6.1.2 Day mode 2 (Demand-Driven)

Days are sorted by their **total demand**, i.e. $\sum_{tc} \theta_{dtc}$ (the total number of required employee-slots, that is, of half-hours of work), in descending order, with random tie-breaking. The intuition is to let the days with the largest absolute workload pick from the full pool of employees before lighter days consume availability.

6.1.3 Day mode 3 (Bottleneck-Driven)

Days are sorted in descending order of the **bottleneck score**, again with random tie-breaking. The score is computed once, before any assignment, from the initial minimum demand. Unlike Mode 2, it does not look at raw demand but at the *ratio* between the total demand of each competence on that day and the number of employees available for that competence on that day. This gives priority to days that are tight on labour supply even when their absolute demand is modest.

$$\text{bottleneck}(d) = \sum_{c \in C} \frac{demand(dc)}{\max\big(available(dc),\, 1\big)} \tag{2}$$

where $demand(dc) = \sum_t \theta_{dtc}$ and $available(dc)$ is the number of employees e with $c \in assignableCompetence_e$ whose mode on day d is not `closed`, `unavailable` or `preferred_day_off`.

6.2 Priority Weight Evaluator

This function converts a $(competence, level)$ pair into its arithmetic priority weight $p_{c\ell}$ (Table 2), by locating the coverage-priority tier that the pair falls into and returning $p_{c\ell} = 1 + (priority - 1) \cdot 10$. Lower values represent higher-priority competence/level combinations (e.g. Management tiers outrank generic `Employees` tiers). This weight is the sole quantity behind **Objective 2**.

6.3 Best Competence for Slot

This function answers, for one employee, day, and slot, the question *"if this employee works this slot, which of their competences should they perform?"*. Among $assignableCompetence_e$, it picks the competence with the largest remaining shortage ($need_{dtc}$), breaking ties first by the lowest priority weight $p_{c\ell}$ (Section 6.2), and finally by the rarest competence, so that a versatile employee is not wasted on a competence that other, less flexible colleagues could equally well cover.

7 Pseudocode

Algorithm 1 `Minimuns(day_order_mode)` – Greedy block construction

for each $d \in OD$ **do**
  for each $e \in OE_d$ **do**
    if $mode_{ed} \in \{\texttt{closed}, \texttt{unavailable}, \texttt{preferred_day_off}\}$ **then**
      $block_{e,d} \leftarrow \texttt{None}$; **continue**
    end if
    for each $b \in B_{ed}$ **do**
      $(\text{coverage}, \text{cost}) \leftarrow$ _SCORE_BLOCK(edb)
      $S_{ed} \leftarrow S_{ed} \cup \{(\text{coverage},\ \text{cost},\ rand(),\ b)\}$
    end for
    sort S_{ed} by $(-\text{coverage},\ \text{cost},\ rand)$ ascending
    for each $(\cdot, \cdot, \cdot, b) \in S_{ed}$ **do**
      if _REST_OK(e, d, b) **then**
        $applied \leftarrow$ _APPLY_BLOCK(e, d, b)
        break
      end if
    end for
    if not *applied* **then**
      $block_{ed} \leftarrow \texttt{None}$;
    end if
  end for
end for